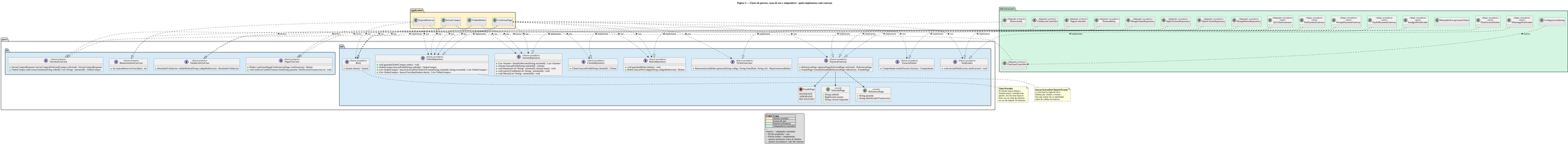

Página 3 — Clases de puertos, casos de uso y adaptadores : quién implementa cada contrato

application
ExpirarReservas
IniciarCompra
ValidarBoleto
ConfirmarPago
ports
in
«Puerto primario» CheckoutUseCase
IniciarCompraResponse iniciarCompra(SolicitudCompra solicitud) : IniciarCompraResponse
OrdenCompra seleccionarAsientos(String ordenId, List~String~ asientosId) : OrdenCompra
«Puerto primario» MantenimientoUseCase
int expirarReservasVencidas() : int
«Puerto primario» ValidacionUseCase
ResultadoValidacion validarBoleto(String codigoRedencion) : ResultadoValidacion
«Puerto primario» PagosUseCase
Boleto confirmarPago(ConfirmacionPago confirmacion) : Boleto
void notificarCambioTransaccion(String pasarela, NotificacionTransaccion n) : void
out
«Puerto secundario» Reloj
Instant ahora() : Instant
«Puerto secundario» OrdenRepository
void guardar(OrdenCompra orden) : void
OrdenCompra buscarPorId(String ordenId) : OrdenCompra
List~OrdenCompra~ buscarActivasPorClienteYEvento(String clienteId, String eventoId) : List~OrdenCompra~
List~OrdenCompra~ buscarVencidas(Instant ahora) : List~OrdenCompra~
«Puerto secundario» AsientoRepository
List~Asiento~ listarPorRecinto(String recintoId) : List~Asiento~
Asiento buscarPorId(String asientoId) : Asiento
void bloquear(List~String~ asientosId, Instant hasta) : void
void marcarVendidos(List~String~ asientosId) : void
void liberar(List~String~ asientosId) : void
«Puerto secundario» ClienteRepository
Cliente buscarPorId(String clienteId) : Cliente
«Puerto secundario» BoletoRepository
void guardar(Boleto boleto) : void
Boleto buscarPorCodigo(String codigoRedencion) : Boleto
«Puerto secundario» TicketGenerator
RepresentacionBoleto generar(String codigo, String firmaHash, String url) : RepresentacionBoleto
«Puerto secundario» PaymentGateway
ReferenciaPago registrarPago(SolicitudPago solicitud) : ReferenciaPago
EstadoPago consultarEstado(ReferenciaPago referencia) : EstadoPago
«Puerto secundario» FacturaEmitter
Comprobante emitir(Factura factura) : Comprobante
«Puerto secundario» Notificador
void enviar(Notificacion notificacion) : void
EstadoPago
PENDIENTE
APROBADO
RECHAZADO
«record» SolicitudPago
String ordenId
BigDecimal monto
String correoComprador
«record» ReferenciaPago
String pasarela
String identificadorTransaccion
infrastructure
«Adaptador primario» ReservasJob
«Adaptador primario» ValidacionController
«Adaptador primario» PagosController
«Adaptador secundario» SistemaReloj
«Adaptador secundario» MongoOrdenRepository
«Adaptador secundario» SqliteAsientoRepository
«Adaptador secundario» SqliteClienteRepository
«Adaptador secundario» MongoBoletoRepository
«Adaptador secundario» «mock» QrTicketGenerator
«Adapt. secundario» «mock» PsePaymentGateway
«Adapt. secundario» «mock» WompiPaymentGateway
«Adapt. secundario» «mock» PayPalPaymentGateway
«Adapt. secundario» «mock» SendgridNotificador
ManejadorExcepcionesGlobal
«Adapt. secundario» «mock» DianFacturaEmitter
«Adapt. secundario» «mock» WhatsappNotificador
ConfiguracionSpring
«Adaptador primario» CheckoutController
invoca
usa
implementa
inyecta

Time Provider
El núcleo nunca llama a
Instant.now(): consulta este
puerto. Así los tests fijan la
hora con un reloj de mentira
en vez de esperar 10 minutos.

buscarActivasPorClienteYEvento
La necesita la regla de las 6
boletas por cliente y evento:
hay que sumar las ya aprobadas
antes de validar las nuevas.

Color	Capa
🟨	Nucleo (inside)
🟨	Casos de uso
🟦	Puertos (frontera)
🟩	Adaptadores (outside)

--
«mock» = adaptador simulado
-- Flecha punteada = usa
-- Flecha solida = implementa
-- «puerto primario» entra al sistema
-- «puerto secundario» sale del sistema